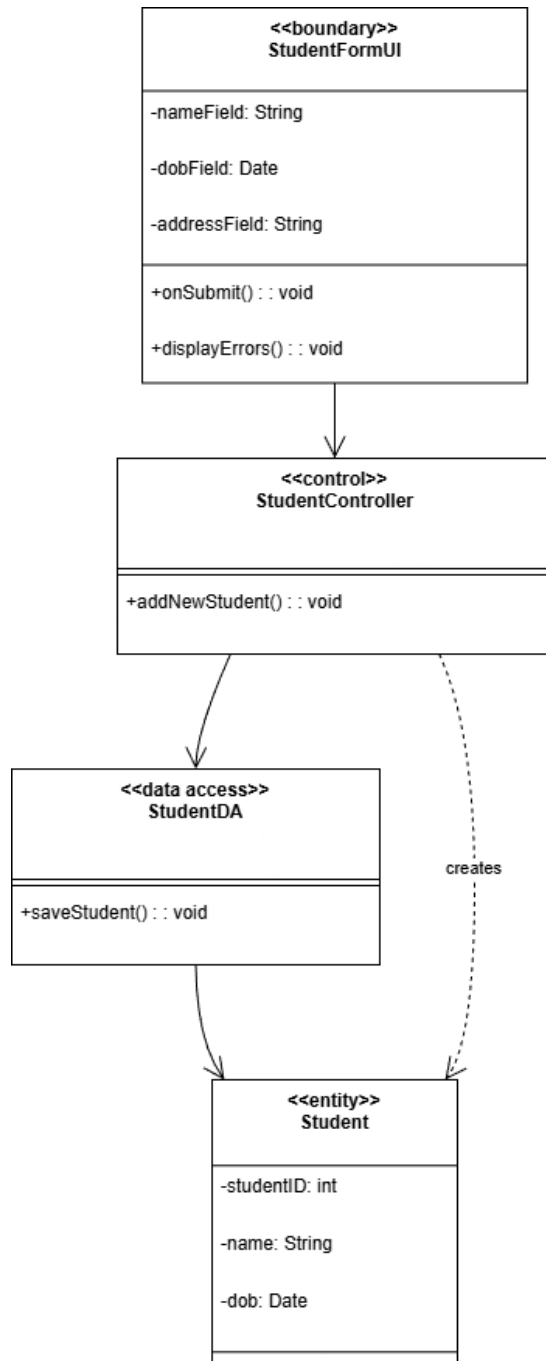


Object-Oriented Design

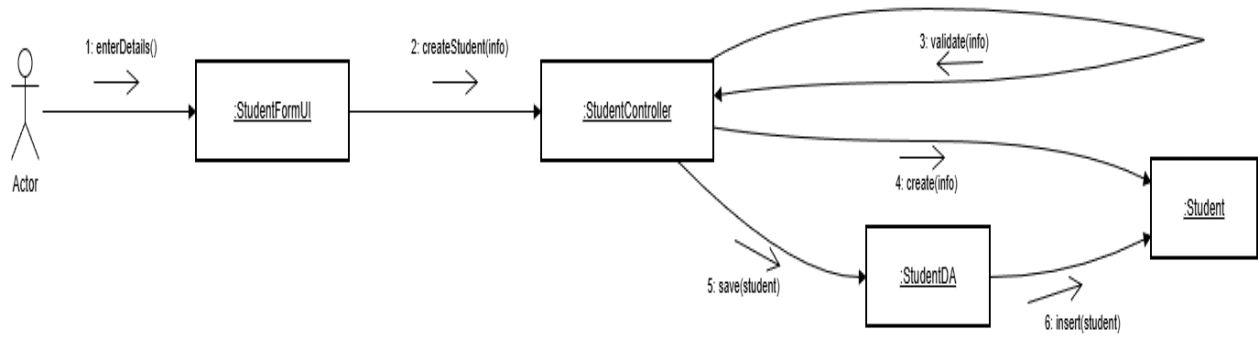
1. Use Case Name: Add Student Info (Moderate)

- **Scenario:** Admin manually enters details to create a new student profile in the system.
- **Triggering event:** Admin submits the "New Student" form.
- **Brief description:** Validate input data (Name, Email, etc.) and create a new Student entity in the database.
- **Actors:** Primary: Admin. Secondary: System.
- **Stakeholders:** Admin; Registrar; Students.
- **Related use cases:** Manage User Accounts.
- **Preconditions:**
 - Admin is authenticated and logged in.
 - The student does not already exist (checked by National ID or Email).
- **Postconditions:**
 - A new Student record is created in the database.
 - The system generates a unique studentID.
- **Flow of activities:**
 - Admin selects "Add Student" and enters student details (Name, Email, Enrollment Date).
 - System (StudentForm) captures the input.
 - System (StudentController) validates that all required fields are filled and email format is correct.
 - System (StudentController) requests creation of a new Student entity.
 - System (Student) creates a new instance and saves data.
 - The system confirms the success and sends a notification to the Admin.
- **Exception conditions:**
 - Validation fails (empty fields) → Show error message.
 - Duplicate email found → Reject and notify Admin.

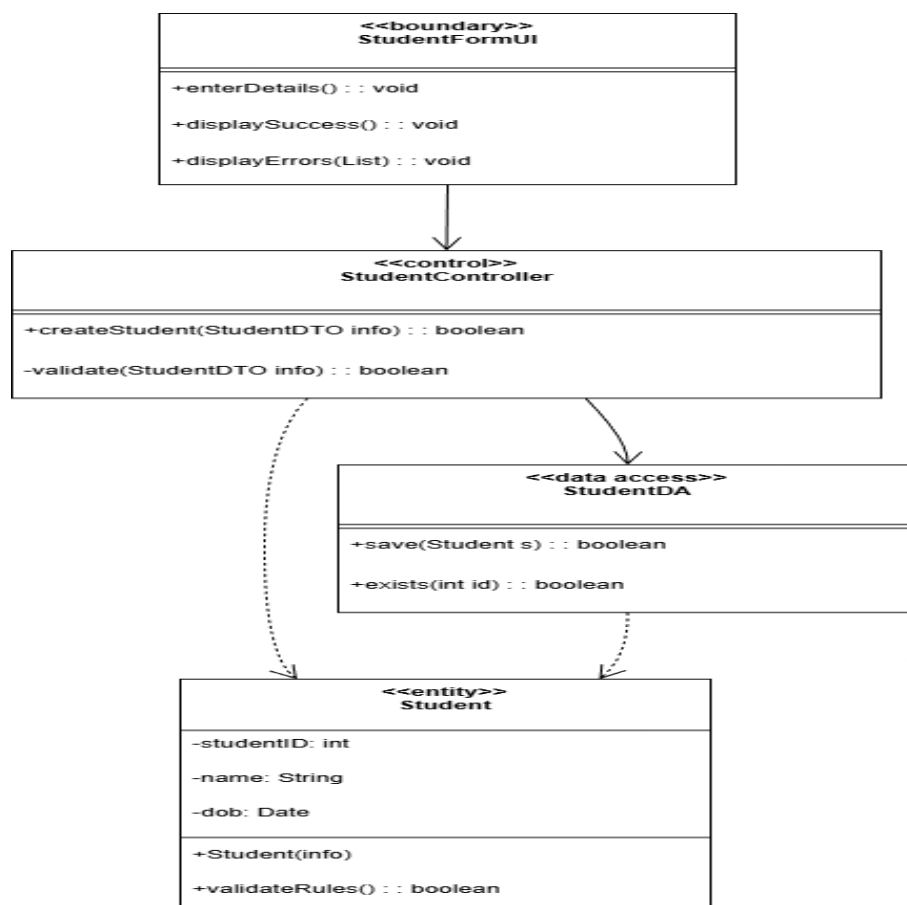
First Cut Design Class Diagram :



Communication Diagram :



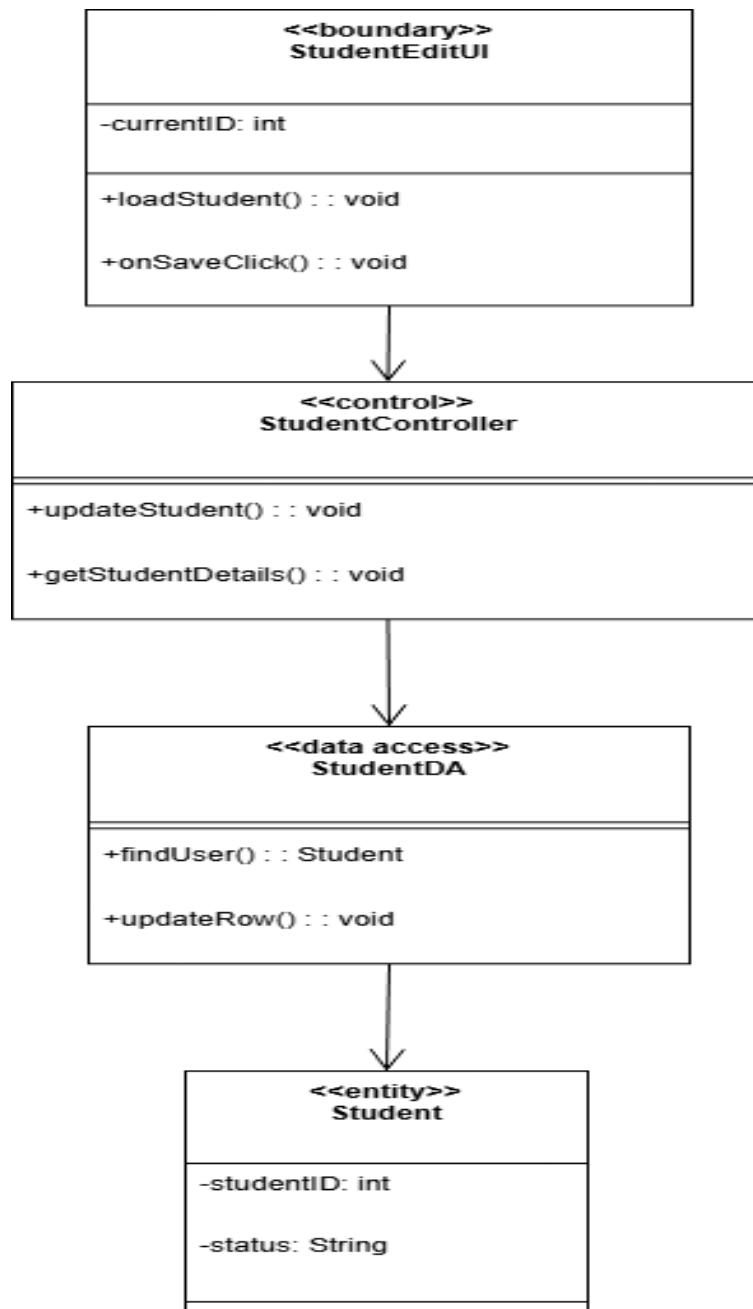
Updated Diagram:



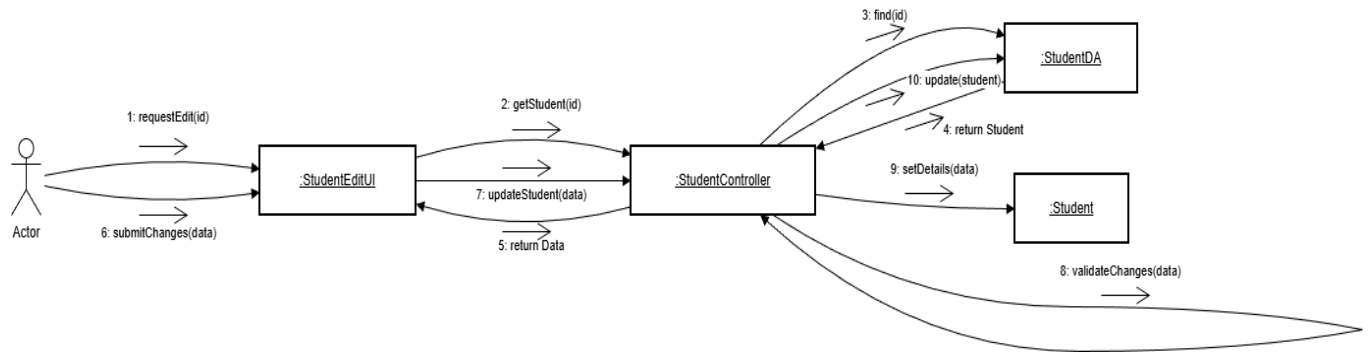
2. Use Case Name: Edit Student Info (Moderate)

- **Scenario:** Admin updates the email or name of an existing student.
- **Triggering event:** Admin saves changes to an existing student profile.
- **Brief description:** Find specific student records, validate changes, and update the entity data.
- **Actors:** Primary: Admin.
- **Stakeholders:** Admin; Student.
- **Related use cases:** Add Student Info; View Grades.
- **Preconditions:**
 - Admin is authenticated.
 - Student records exist in the system.
- **Postconditions:**
 - Student attributes are updated permanently in the database.
- **Flow of activities:**
 - Admin searches for a student, selects "Edit," and modifies fields (e.g., changes Email).
 - System (EditStudentView) captures the updated data.
 - System (StudentController) validates the new data format.
 - System (StudentController) finds the specific Student object by ID.
 - System (Student) updates its details with the new values.
 - The system confirms the update to the Admin.
- **Exception conditions:**
 - Invalid data format → Reject update.
 - Student ID not found in database → Show "Record not found" error.

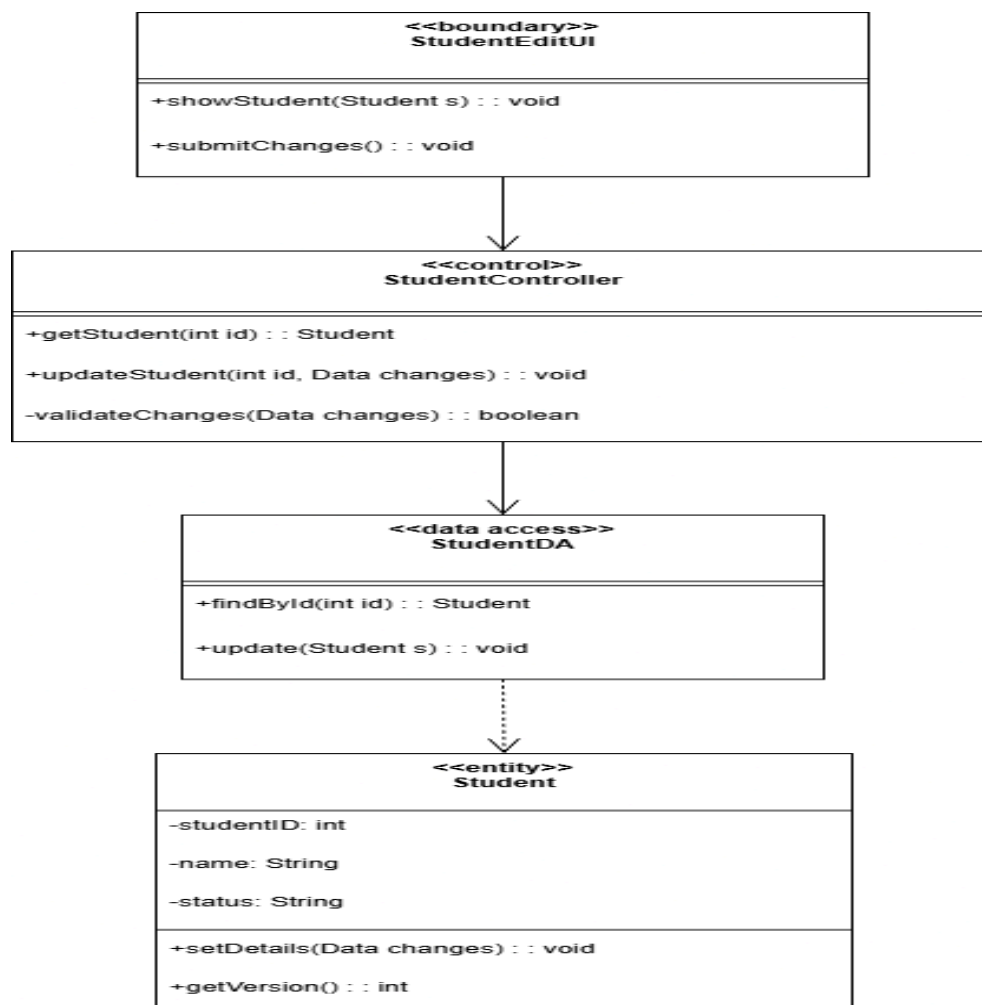
First Cut Design Class Diagram :



Communication Diagram :



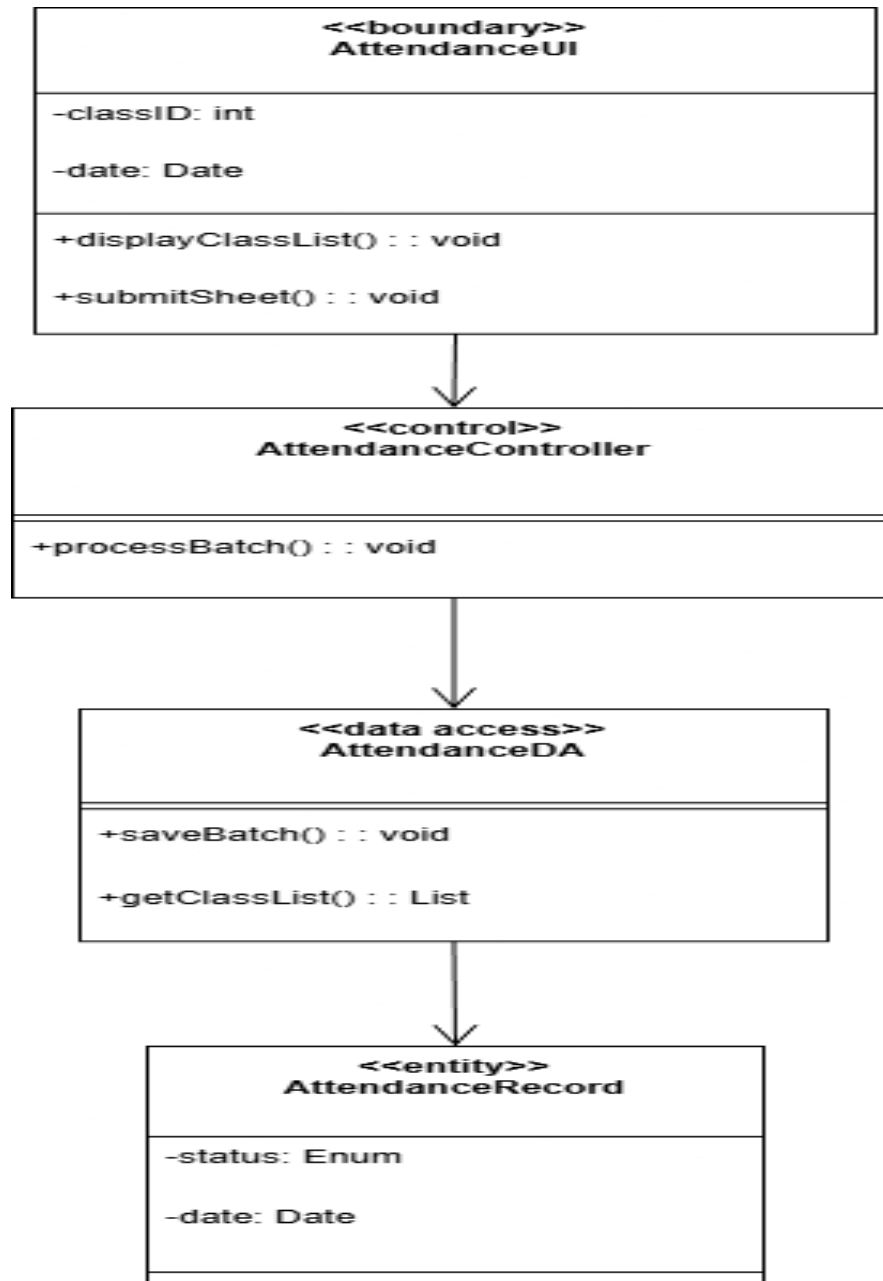
Updated Diagram :



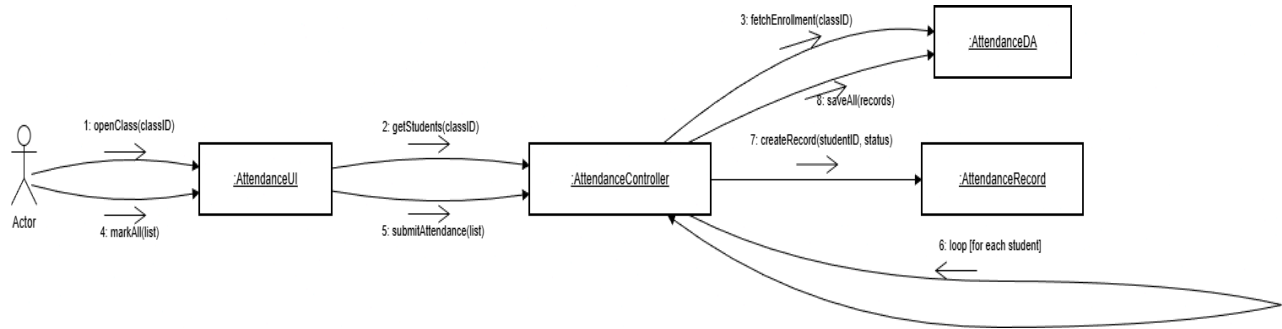
3. Use Case Name: Record Attendance (Moderate)

- **Scenario:** Teacher marks a student as "Present" or "Absent" for a specific class date.
- **Triggering event:** Teacher submits the attendance sheet.
- **Brief description:** Create a new attendance log entry linking a student to a status and date.
- **Actors:** Primary: Teacher. Secondary: Parent (notified).
- **Stakeholders:** Teacher; Student; Parent; Admin.
- **Related use cases:** View Attendance.
- **Preconditions:**
 - The teacher is assigned to the class.
 - The class session is valid and active.
- **Postconditions:**
 - A new AttendanceLog object is created.
 - Log is linked to the Student.
- **Flow of activities:**
 - The teacher selects a class/date and marks status for a student (Present/Absent).
 - System (AttendanceView) submits the data.
 - System (AttendanceController) receives the request and validates the session.
 - System (AttendanceController) initiates creation of a log entry.
 - System (AttendanceLog) creates itself with date and status.
 - System (Student) links the new log to its record.
- **Exception conditions:**
 - Database connection failure → Notify Teacher to try again.
 - Attendance already recorded for this date → Prompt to overwrite or cancel.

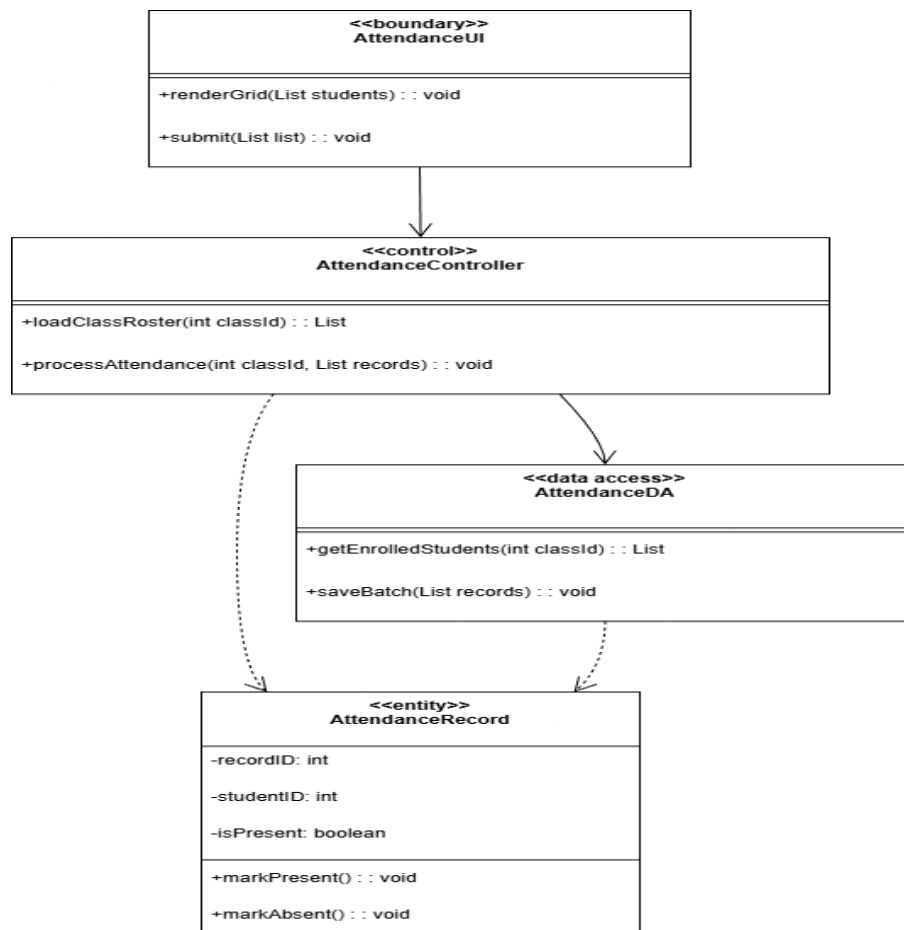
First Cut Design Class Diagram :



Communication Diagram :



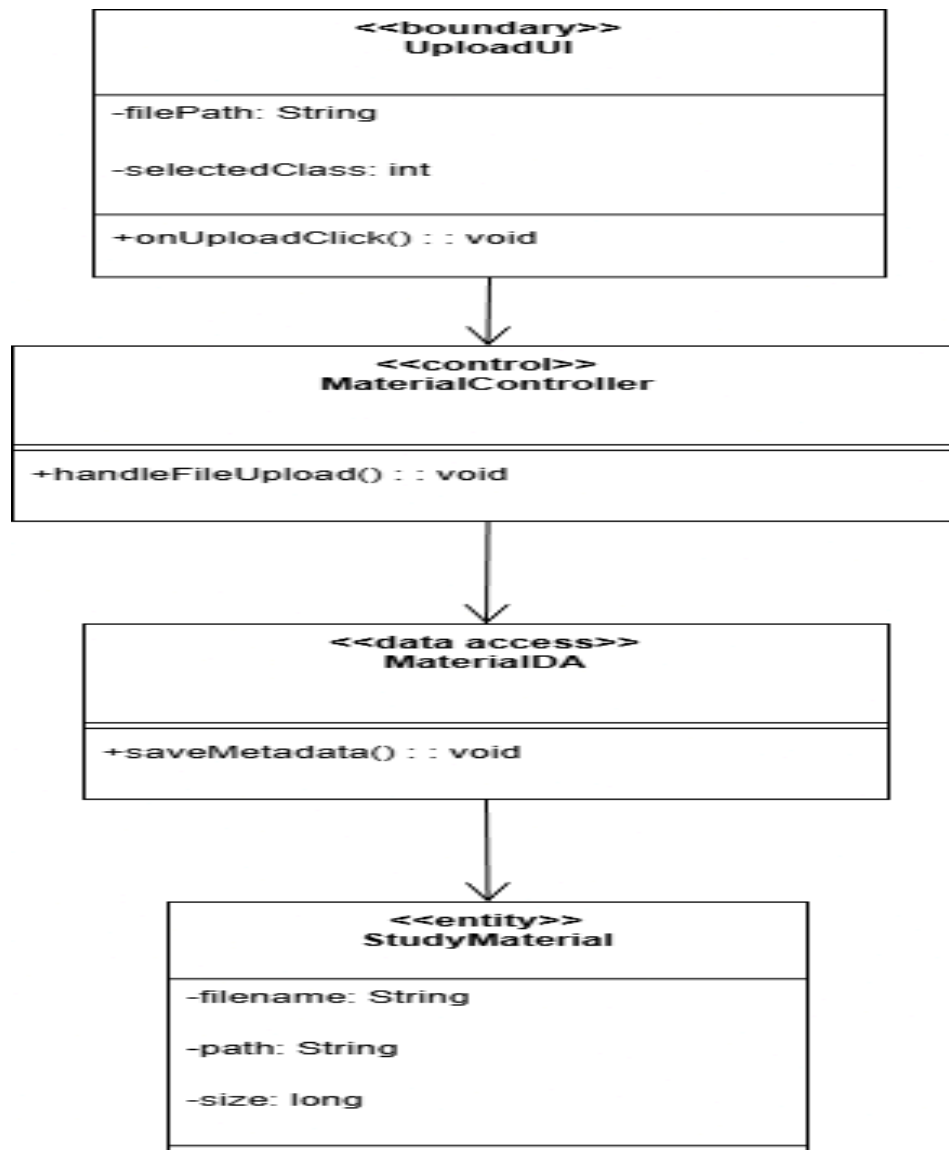
Updated Diagram :



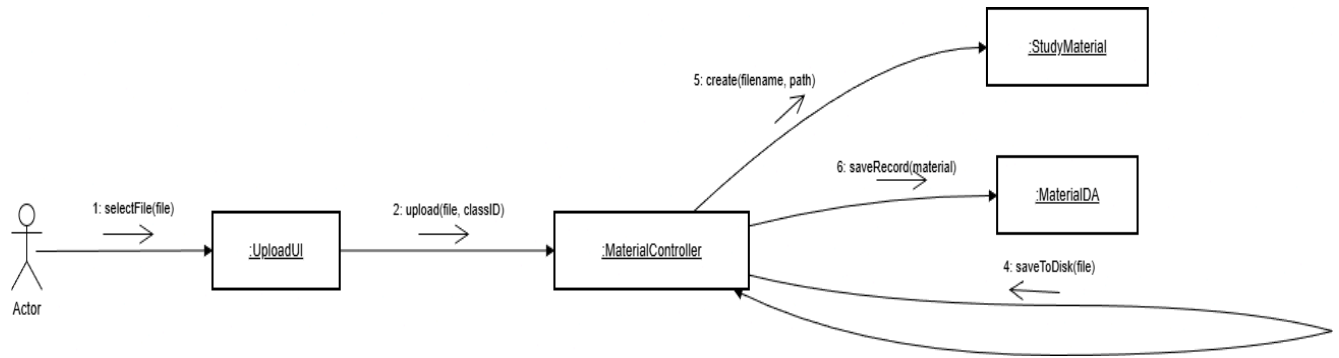
4. Use Case Name: Upload Study Materials (Moderate)

- **Scenario:** Teacher uploads a PDF or resource file for students to access.
- **Triggering event:** Teacher selects a file and clicks upload.
- **Brief description:** Handle binary file upload, save path to database, and create Material entity.
- **Actors:** Primary: Teacher. Secondary: Student (viewer).
- **Stakeholders:** Teacher; Student.
- **Related use cases:** Access Study Materials.
- **Preconditions:**
 - The teacher is logged in.
 - File format is supported (PDF, DOCX, JPG).
- **Postconditions:**
 - Study Material object created.
 - File stored in server/cloud storage.
- **Flow of activities:**
 - The teacher selects a file from their device, enters a title, and clicks "Confirm".
 - System (UploadView) sends file data to the controller.
 - System (MaterialController) validates file type and size.
 - System (MaterialController) initiates creation of the material record.
 - System (StudyMaterial) creates the object and sets the upload date to the current timestamp.
 - System confirms upload completion.
- **Exception conditions:**
 - File too large or wrong format → Show error message.

First Cut Design Class Diagram :



Communication Diagram :



Updated Diagram :

